

Pre-Install Behavioral Analysis for npm Supply Chain Defense

Abstract

npm supply chain attacks execute arbitrary code on developer machines and CI pipelines before anyone reviews the package. The attacks are zero-day by nature. A freshly published malicious package has no CVE, no advisory, no signature in any threat database.

Existing defenses (npm audit, Snyk, Dependabot) are reactive: they match packages against known vulnerability databases and catch nothing that hasn't been reported yet.

This paper presents a behavioral pre-install analysis approach that intercepts package manager commands, performs dry-run resolution to determine what would change, downloads and inspects package contents, and runs six parallel risk analyzers against every added or updated dependency. Execution is gated behind policy evaluation and explicit approval.

The approach catches install-script attacks, typosquatting, dependency confusion, maintainer compromise, obfuscated code, manifest confusion, and trojan transitive dependencies, all without relying on any external vulnerability database. Every decision is recorded in a tamper-resistant HMAC-chained audit log.

The implementation (Nighthawk) runs the full analysis pipeline in under 3 seconds on a project with 200 dependencies. It has been validated with 331 tests, including 17 end-to-end tests against crafted attack packages simulating all major supply chain attack categories.

1. The Defense Gap

The JavaScript ecosystem has a supply chain problem that its tooling doesn't address.

In September 2025, attackers phished a single npm maintainer and published malicious updates to 18 packages including `chalk` and `debug`, which have a combined 2+ billion weekly downloads. The payload was a self-propagating worm that iterated through the maintainer's other packages, injected code, and auto-republished. CISA issued a national advisory. The malicious code was live for approximately 2 hours.

This was not the first time:

- `event-stream` (2018): social engineering a burned-out maintainer into handing over publish access.
- `ua-parser-js` (2021): account hijacking of a package with 7 million weekly downloads, malicious versions live for 4 hours.
- `colors.js` (2022): the legitimate maintainer sabotaged 3.3 billion lifetime downloads in protest.
- `node-ipc` (2022): protestware that overwrote files on machines with Russian IP addresses.

These are not edge cases. They are the dominant threat model for JavaScript dependency management.

The standard defense is `npm audit`, which checks your dependency tree against the GitHub Advisory Database. The problem is structural: `npm audit` can only find vulnerabilities that have been discovered, reported, triaged, assigned a CVE, and added to the database.

Supply chain attacks are zero-day by definition. The malicious code is new. By the time an advisory exists, the attack window is closed and the damage is done. The September 2025 attack was over in 2 hours. CVE databases take days to weeks.

Tools like `Socket.dev` perform behavioral analysis but operate as external services. There is, to my knowledge, no open-source CLI tool that performs behavioral pre-install analysis locally, runs the full detection pipeline before any code executes, works offline, and produces a cryptographically verifiable audit trail of every decision.

2. Attack Taxonomy

Testing against real-world attack patterns and simulated attack packages reveals five distinct categories that a pre-install defense must handle. Each exploits a different part of the `npm install` pipeline.

Category 1: Install Script Injection. The package declares a `postinstall` (or `preinstall`, `install`, `prepare`) lifecycle script that runs arbitrary code at install time. This is the primary delivery mechanism. The malicious code executes automatically, with the developer's full privileges, before anyone looks at the package.

The `event-stream`, `ua-parser-js`, and September 2025 attacks all used this vector. `npm install` runs lifecycle scripts for every package in the tree, including transitive dependencies.

Category 2: Typosquatting. The attacker registers a package name that is one or two characters away from a popular package (`lodashs` for `lodash` , `expres` for `express`).

A variant uses scoped packages to impersonate popular unscoped packages.

`@attacker/lodash` uses the legitimate name under a different scope, exploiting confusion about package identity. Developers make typos. Automated scripts have misspelled package names. This attack has been documented since at least 2016, when a research demonstration infected 17,000 machines across npm, PyPI, and RubyGems.

Category 3: Account Hijacking / Maintainer Compromise. The attacker gains control of a legitimate maintainer's account (credential stuffing, phishing, token theft) and publishes a malicious patch or minor version.

From the outside, the package looks identical: same name, same maintainer, semver-compatible version bump. The September 2025 attack specifically targeted maintainers with many packages to maximize blast radius.

Category 4: Dependency Confusion. The attacker publishes a public package with the same name as an organization's private/internal package, using a higher version number. Package managers that check the public registry before (or alongside) the private registry pull the attacker's code instead.

First publicly demonstrated by Alex Birsan in 2021 against Apple, Microsoft, and dozens of other companies.

Category 5: Trojan Transitive Dependencies. Rather than attacking a popular package directly, the attacker targets a small, obscure package three or more levels deep in the dependency tree.

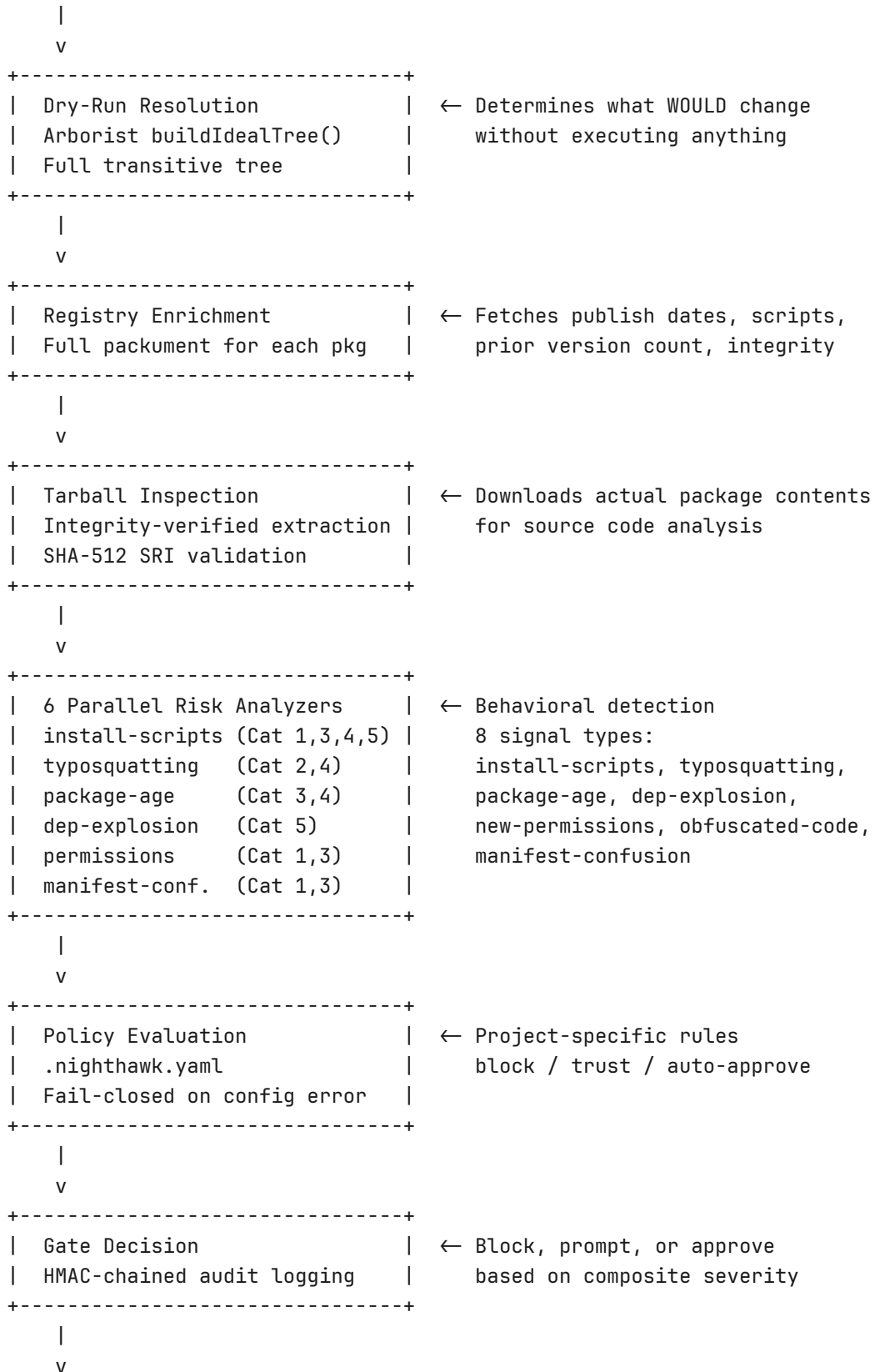
The average npm project has 300-1,200 transitive dependencies. Nobody reviews them all. A malicious `postinstall` in a package you've never heard of, buried four levels deep, runs with the same privileges as code you wrote yourself.

Categories 1-5 account for the dominant real-world attack patterns observed in the npm ecosystem between 2018 and 2025.

3. The Pre-Install Analysis Pipeline

Six mechanisms, each addressing different attack categories:

Developer runs: nighthawk install express



```
Real install (only if approved)
  |
  v
Post-install verification (did installed packages match prediction?)
```

3.1 Dry-Run Resolution

The critical insight is that npm's dependency resolver (`@npmcli/arborist`) can compute the full ideal dependency tree without writing to disk or executing any scripts. Calling `buildIdealTree({ add: ['express'] })` on an Arborist instance returns the complete transitive tree as an in-memory data structure: every package, every version, every resolved tarball URL.

This gives the analysis pipeline visibility into the full blast radius of an install command before anything executes. A developer running `nighthawk install express` sees not just `express`, but every transitive dependency that would be added, updated, or removed.

The dry-run adds zero risk. No filesystem writes, no script execution, no side effects.

For Bun, the equivalent is `bun add --dry-run`, which outputs the resolution plan as text. The output is parsed into the same normalized format.

3.2 Registry Enrichment

For each added or updated package, the pipeline fetches the full packument (not the abbreviated install metadata) from the npm registry. The full packument includes:

- **time map**: when each version was published, enabling package-age analysis
- **scripts field per version**: lifecycle scripts declared in the packument, enabling install-script detection without downloading the tarball
- **dist.integrity**: SHA-512 hash for tarball verification
- **Prior version count**: derived from the `time map`, distinguishing established packages from brand-new ones

Packument data is cached immutably per version (a published version's metadata never changes). Cache files are HMAC-signed to prevent tampering.

3.3 Tarball Inspection

Package tarballs are downloaded, integrity-verified against the SRI hash from the packument, and extracted to a temporary directory for source code analysis.

The permissions analyzer scans actual `.js / .ts` files in the tarball for dangerous module usage. This catches cases where the packument metadata doesn't reveal the full picture. A package can use `child_process` in its source code without declaring it anywhere in `package.json`.

Downloads run in parallel with a concurrency limit of 8.

3.4 The Six Analyzers

All analyzers run in parallel via `Promise.all`. Errors in any analyzer produce a LOW-severity signal rather than crashing the pipeline.

Install Scripts. Detects `preinstall`, `install`, `postinstall`, `prepare`, `prepublish`, and `prepack` lifecycle scripts. An allowlist covers packages that legitimately need scripts (esbuild, sharp, husky, node-gyp wrappers).

Unknown packages with lifecycle scripts receive BLOCK severity, the strongest possible signal. This is the single most impactful analyzer because install scripts are the dominant delivery mechanism.

Typosquatting. Computes Levenshtein edit distance between the package name and a list of popular npm packages. Edit distance 1 = HIGH, edit distance 2 = MEDIUM.

For scoped packages (`@scope/name`), the analyzer extracts the bare package name and compares it against top packages. `@attacker/lodash` is flagged as HIGH because the bare name matches a top package exactly, regardless of scope. This catches scoped squatting and dependency confusion variants.

The implementation uses a two-row sliding matrix for $O(\min(m,n))$ space with a length pre-filter that skips comparisons when the length difference exceeds 2.

Package Age. Checks the publish timestamp and prior version count from the registry enrichment step. A package published less than 24 hours ago with zero prior versions = HIGH. Less than 7 days with zero prior versions = MEDIUM.

This catches freshly-created attack packages but doesn't flag established packages with long histories.

Dependency Explosion. Counts the number of new transitive dependencies an install would add. More than 100 = HIGH. More than 50 = MEDIUM.

This flags packages that pull in unusually large trees, a potential indicator of trojan transitive attacks or packages designed to maximize attack surface.

Permissions. Scans tarball source files for `require('fs')`, `require('net')`, `require('child_process')` and their ESM/ node: prefix variants. Also detects obfuscation patterns: string concatenation (`require('child' + '_process')`), `eval()`, `new Function()`, dynamic `require()` and `import()` calls with non-literal arguments, `String.fromCharCode()`, hex escape sequences, and bracket-notation property access on `process`.

Compares against the prior version's cached scan to detect newly-added dangerous capabilities. The permissions analyzer emits two distinct signal types: `new-permissions` (MEDIUM) for standard module imports, and `obfuscated-code` (HIGH) for evasion techniques.

The severity difference reflects that obfuscation is a deliberate attempt to bypass detection. A utility library that starts importing `child_process` in a new version is suspicious regardless of whether it has a CVE. One that starts using `eval()` is more so.

Manifest Confusion. Compares the `package.json` inside the extracted tarball against the registry packument metadata. The npm registry does not validate that tarball contents match the manifest, and over 800 packages have been found with mismatches.

The analyzer detects name mismatches (tarball claims to be a different package), version mismatches, hidden lifecycle scripts (present in tarball but absent from the packument), and hidden dependencies.

Any mismatch is flagged at HIGH severity because manifest confusion is deliberate deception. There is no benign reason for a tarball's identity to differ from what the registry advertises.

3.5 Multiplicative Severity

Individual signals combine through multiplicative escalation:

Condition	Composite Severity
Any BLOCK signal	BLOCK
2+ HIGH signals	BLOCK
1 HIGH + 1 MEDIUM	HIGH
2+ MEDIUM signals	HIGH
1 HIGH alone	HIGH
1 MEDIUM alone	MEDIUM

A brand-new package (age: HIGH) with a `postinstall` script (install-scripts: BLOCK) gets BLOCK composite. A `typosquat` (HIGH) that also adds `child_process` usage (permissions: MEDIUM) escalates to HIGH.

The escalation is deliberate. Combinations of suspicious signals are more dangerous than any single signal.

3.6 Policy Evaluation

Projects define rules in `.nighthawk.yaml` with a strict precedence chain:

1. **Block list:** absolute veto, always blocked regardless of analysis
2. **Allow-all:** full bypass for known-good packages
3. **Scoped trust:** filter known signals for trusted scopes (e.g., `@mycompany/*`)
4. **Auto-approve rules:** approve if ALL criteria match (semver range, minimum age, minimum downloads)
5. **Severity threshold:** in CI, HIGH = block; in interactive, HIGH = prompt for user decision

Policy configuration errors (bad YAML, schema violations) cause the tool to exit with code 2. This is fail-closed by design. A broken config blocks everything rather than silently allowing everything.

3.7 Tamper-Resistant Audit Trail

Every gate decision is recorded in `~/.nighthawk/audit.jsonl` as an HMAC-chained log. Each entry's HMAC covers the entry fields plus the previous entry's HMAC, creating a cryptographic chain. A sentinel file tracks entry count and the last HMAC, detecting log truncation.

The HMAC key is per-machine, stored with `0600` permissions, and all comparisons use `crypto.timingSafeEqual` to prevent timing attacks.

The audit log answers: what was installed, when, what signals were detected, and what the decision was. The cryptographic chain provides proof that the log hasn't been tampered with after the fact.

4. Results

4.1 Detection Coverage

E2E testing against crafted attack packages covering all five attack categories:

Attack Scenario	Packages	Signals Fired	Exit Code
Install script (postinstall exfiltration)	postinstall-exfil@1.0.0	install-scripts: BLOCK	1 (blocked)
Install script (preinstall reverse shell)	preinstall-reverse-shell@1.0.0	install-scripts: BLOCK	1 (blocked)
Typosquatting	typosquat-lodahs@1.0.0	typosquatting: HIGH, package-age: HIGH	1 (blocked)
Dependency explosion	dep-explosion@1.1.0 (52 deps)	dep-explosion: MEDIUM, package-age: HIGH	1 (blocked)
New permissions	new-permissions@1.1.0	permissions: MEDIUM, package-age: HIGH	1 (blocked)
Hijacked maintainer (clean version)	hijacked-maintainer@1.0.0	none	0 (approved)
Hijacked maintainer (compromised version)	hijacked-maintainer@1.0.1	install-scripts: BLOCK	1 (blocked)
Dependency confusion (squatter)	acme-internal-utils@99.0.0	install-scripts: BLOCK	1 (blocked)
Trojan transitive (3 levels deep)	popular-package@2.0.0	install-scripts: BLOCK (via trojan-level3)	1 (blocked)
Install script chain	script-chain-a@1.0.0	install-scripts: BLOCK (via chain-b)	1 (blocked)
Clean package (false positive test)	clean-package@1.0.0	none	0 (approved)

17 E2E test cases (11 attack/detection scenarios + 6 false-positive edge cases). Zero false negatives on attack packages, zero false positives on clean packages.

The false-positive edge cases validate that established packages with permissions, allowlisted build tools, version updates with prior history, boundary dependency counts, and scoped internal packages all pass without blocking. This is critical because false positives kill adoption faster than missed attacks.

The trojan transitive test is notable: the malicious `postinstall` is in `trojan-level3`, three dependency levels below the package the developer asked to install. Arborist's full tree resolution surfaces it.

Beyond E2E, the full test suite comprises 331 tests across 45 files covering analyzer unit tests (118), policy evaluation (59), package manager integration (62), storage integrity (21), lockfile parsing (18), scoring validation (16), and CLI commands (17).

4.2 Performance

The analysis pipeline targets under 3 seconds of added latency on a project with 200 dependencies. In practice:

- Mock registry E2E suite (17 tests): **4.5 seconds total**
- 200-package analysis: completes without error in scale testing
- 500-package analysis: completes within 15-second time bound
- Per-package analysis: sub-second after registry metadata is cached
- Tarball downloads: parallel with concurrency 8, integrity-verified

The dry-run step (Arborist `buildIdealTree`) is the dominant cost. On projects with large trees, this is the same resolution npm would do anyway. Nighthawk adds the analysis overhead on top, not the resolution overhead.

4.3 What It Doesn't Catch

This approach has clear limitations.

Obfuscated code. The permissions analyzer detects a broad set of obfuscation techniques: `eval()`, `new Function()`, `dynamic require()` with non-literal arguments, string concatenation (`require('child' + '_process')`), `String.fromCharCode()` character assembly, `dynamic import()` with non-literal arguments, hex/unicode escape sequences, and bracket-notation property access on `process` (e.g. `process["env"]`). These are flagged as `obfuscated-code` signals at HIGH severity.

However, the detection is regex-based, not AST-based. Sophisticated obfuscation (computed property chains built at runtime, WebAssembly modules that call native APIs, or multi-stage payload assembly that spans multiple files) will evade detection. Full AST analysis or entropy-based scanning would close these remaining gaps.

Manifest confusion. The manifest confusion analyzer compares the tarball's `package.json` against the registry packument and detects name, version, script, and dependency mismatches at HIGH severity. However, it does not yet detect all forms of metadata manipulation. For example, a tarball that includes additional undeclared files (binaries, native modules) that aren't referenced in `package.json` at all.

Legitimate new packages. The package-age analyzer flags all brand-new packages with no history. This is by design (new packages are genuinely higher risk), but it means a developer publishing their first package will trigger a HIGH signal. The policy engine's `trust` and `allow-all` rules provide the escape hatch.

Runtime attacks. Nighthawk only analyzes pre-install. A package that passes all static checks but loads malicious code dynamically at runtime (e.g., fetching a payload from a URL on first `require()`) is outside scope. Runtime monitoring is a different problem with different tradeoffs.

Allowlist maintenance. The install-scripts analyzer uses a static allowlist for packages that legitimately need lifecycle scripts. This list needs ongoing curation as the ecosystem evolves. A stale allowlist either blocks legitimate packages or misses new attack patterns.

5. Architecture Decisions

Several design choices shape the approach.

Local-only, no external service. All analysis runs locally. No package hashes are sent to external APIs. No network dependency beyond the npm registry itself. This matters for air-gapped environments, sensitive codebases, and organizations that can't send dependency information to third parties.

Flat files with HMAC over SQLite. The storage layer uses HMAC-signed JSON files instead of a database. This eliminates native dependencies (SQLite requires compilation on some platforms), makes the data inspectable with any text editor, and keeps the tool's own supply chain minimal. The HMAC signing provides tamper resistance without the complexity of a database WAL.

Fail-closed everywhere. Policy config parse errors exit 2 (not silent allow). Tampered cache files are deleted (not used). Broken audit chains block installs until manually resolved. The bias is toward false positives over false negatives. Blocking a legitimate package is recoverable. Allowing a malicious one is not.

Package manager wrapping, not replacing. Nighthawk wraps npm and Bun rather than reimplementing dependency resolution. The dry-run step uses Arborist (npm's own resolver) to guarantee resolution accuracy. This means nighthawk's analysis matches what the real install would do, because it uses the same resolver.

Immutable per-version caching. A published npm version's metadata never changes (the registry enforces this). Risk assessments for a given `name@version` are cached immutably after first analysis. This means repeated installs of the same version incur zero analysis overhead. The cache version counter allows invalidating all caches when the analysis logic changes.

6. Closing Thoughts

The core observation is that npm's security tooling is oriented around known vulnerabilities while the dominant threat (supply chain attacks) produces unknown vulnerabilities.

`npm audit` is necessary but addresses a different threat model than the one that produced `event-stream`, `ua-parser-js`, `colors.js`, and the September 2025 npm heist. There is no CVE for "the maintainer's account was compromised 20 minutes ago."

Behavioral pre-install analysis doesn't require a vulnerability database. It asks: does this package have lifecycle scripts? Is the name suspiciously close to a popular package? Was it published today with no version history? Does it pull in 100 transitive dependencies? Did it just start importing `child_process`? Does the tarball's identity match what the registry advertises?

None of these questions require prior knowledge of the specific attack. They work on day zero.

The six-analyzer pipeline, producing eight distinct signal types through multiplicative escalation, catches all major attack categories tested with zero false positives across seven distinct clean-package edge cases. The policy engine provides the necessary escape hatches for legitimate packages that trigger signals. The HMAC-chained audit log creates a verifiable record for compliance and incident response.

There is significant room to push further. Obfuscation detection now covers `eval()`, `new Function()`, `dynamic require()` and `import()`, string concatenation, `String.fromCharCode()`, hex escape sequences, and `process` bracket-notation access, all at HIGH severity. The next step is AST-based analysis (detecting computed property chains built at runtime, multi-file payload assembly) and entropy-based scanning (flagging files with unusually high Shannon entropy, which correlates with encoded/encrypted payloads). WebAssembly modules that invoke native APIs remain undetected.

Manifest confusion detection is implemented and catches name, version, script, and dependency mismatches at HIGH severity. Provenance verification using npm's Sigstore-based provenance attestations would add a supply chain authenticity signal. Maintainer change detection (new maintainers added, sole maintainer replaced) would flag the social engineering vector that enabled event-stream.

Scoped typosquatting detection (`@attacker/lodash` matching the bare name against top packages) is already implemented and catches the intersection of Category 2 and Category 4 attacks. A runtime monitoring layer, tracking which packages actually access the filesystem, network, or subprocess APIs at runtime, would catch attacks that bypass all static analysis.

The pipeline architecture is designed for these extensions. Each analyzer is a strategy pattern implementation that receives a `PackageChange` and returns `Signal[]`. Adding a new detection capability means adding a new analyzer. The pipeline handles parallel execution, error isolation, composite scoring, and caching automatically.